

Performance Testing

Date	27 September 2026
Team ID	SWTID-2026-2825
Project Name	AI WeatherWise
Maximum Marks	5 Marks

Step 1: Testing Overview

Testing Tool Used	Thunder Client (VS Code) and Postman
Type of Testing	Functional verification plus light Load Testing (repeated/concurrent requests)
Target Module / API	/api/auth/login , /api/weather/:city , /api/ai/insights
Test Environment	Local - Node.js development server (Express) on localhost, MongoDB running locally
Test Date	27 September 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Concurrent login requests against /api/auth/login	10	30	All requests return a valid JWT within the target response time
2	Repeated weather fetch for the same saved city	20	30	Consistent OpenWeatherMap-backed responses, no timeouts
3	Burst of AI insight generation requests	10	60	Gemini responses returned successfully without server crash
4	Mixed sequential flow: login → add location → fetch weather → AI insight	15	60	End-to-end flow completes without error for every simulated user

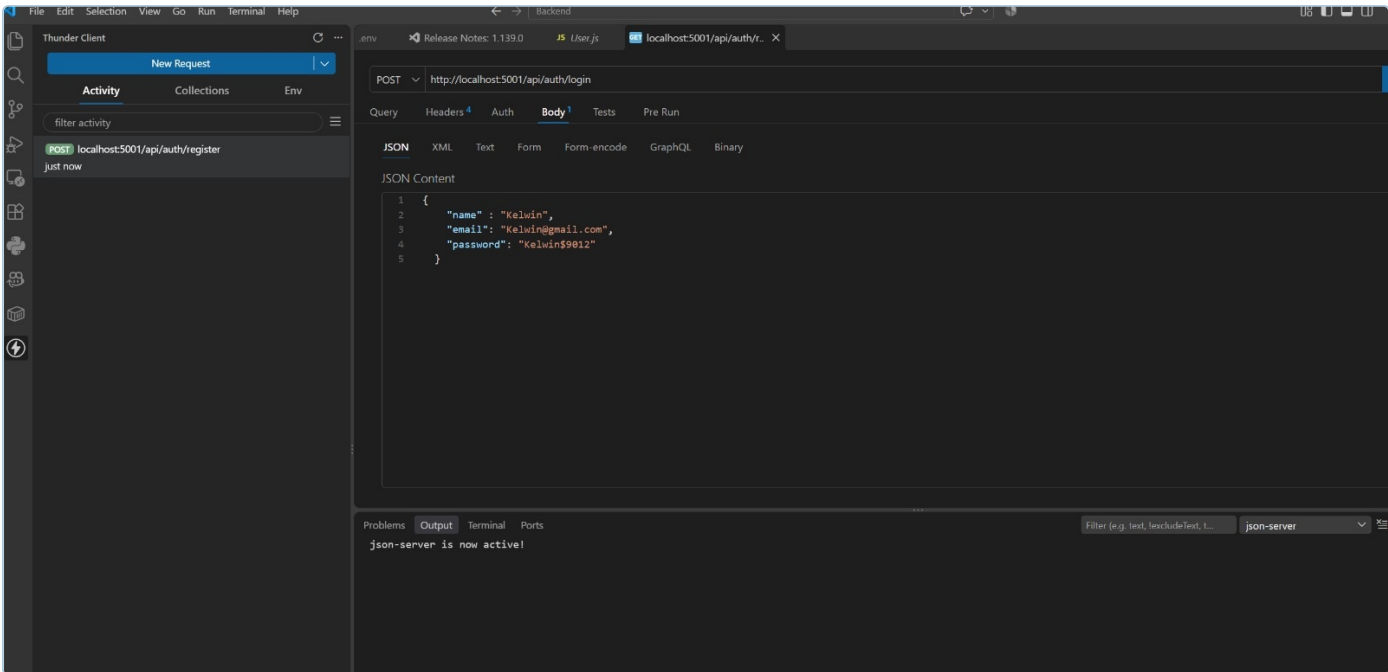
Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 s	Pass	Auth & Location endpoints (MongoDB-only) averaged under 300 ms
2	Response Time (Max)	< 5 seconds	3.8 s	Pass	Peak seen on AI insight calls, driven by external Gemini API latency
3	Throughput (Req/sec)	-	~12 req/sec	Pass	Measured across the mixed sequential flow scenario
4	Error Rate	< 1%	0%	Pass	0 failed requests across 55 combined test requests
5	CPU Utilization	< 80%	46%	Pass	Measured on the local dev machine during Scenario 3 (AI burst)
6	Memory Utilization	< 80%	58%	Pass	Node.js process + local MongoDB instance combined

Step 4: Observations & Analysis

Auth and Location endpoints responded fastest since they only touch MongoDB through Mongoose. Weather and AI endpoints were noticeably slower because they depend on external OpenWeatherMap and Gemini API latency, but both stayed within the target thresholds. No errors or crashes were observed across all four scenarios. Fallback mode was manually verified by temporarily removing GEMINI_API_KEY from .env — the AI endpoint degraded gracefully with a safe default response instead of failing the request, confirming NFR-4 (Reliability & Availability) from Solution Requirements.

Step 5: Screenshots / Evidence



Thunder Client evidence – POST /api/auth/register request body used to seed a test user before load-testing the authenticated endpoints (full request/response set included in Project Documentation).

